

گزارش پروژه ریاضیات گسسته

بخش رمز هیل

HILL CIPHER

A	B	C	D	E	F	G
H	I	A	T	G	H	X

شماره دانشجویی

40424193

نویسنده

پارسا هاشمی سعادت

رمز هیل (Hill Cipher)

در این پروژه رمز هیل را پیاده‌سازی کرده‌ایم. برنامه سه کار اصلی انجام می‌دهد: رمزگذاری با ماتریس کلید، رمزگشایی و پیدا کردن کلید از روی متن اصلی و متن رمز متناظر.

ساختار پروژه

```
hill_project/
├── src/
│   ├── hill_cipher.py
│   ├── utils.py
│   └── __init__.py
├── tests/
│   └── test_hill_project.py
├── data/
│   └── sample_plaintext.txt
├── docs/
│   └── README.md
├── benchmarks/
│   ├── hill_standard_examples.csv
│   ├── hill_performance_by_text_length.csv
│   └── hill_performance_by_text_length.png
├── output/
├── main.py
├── README.md
└── requirements.txt
```

- `src/hill_cipher.py`: محاسبات ماتریسی، رمزگذاری، رمزگشایی و بازیابی کلید
- `src/utils.py`: توابع ساده مربوط به حروف انگلیسی
- `tests/test_hill_project.py`: تست مثال‌های استاندارد، padding و حمله متن معلوم
- `data/sample_plaintext.txt`: متن نمونه برای ورودی فایل
- `benchmarks/hill_standard_examples.csv`: نتیجه مثال‌های استاندارد 2×2 و 3×3
- `benchmarks/hill_performance_by_text_length.csv`: زمان اجرا و نرخ موفقیت برای طول‌های مختلف متن
- `benchmarks/hill_performance_by_text_length.png`: نمودار زمان و موفقیت benchmark
- `output/`: محل ذخیره خروجی‌های اجرای برنامه

نصب و اجرا

داخل پوشه `hill_project` این دستورها را اجرا کنید:

```
python -m pip install -r requirements.txt
python main.py
```

اگر `main.py` را بدون آرگومان اجرا کنید، همین سه عملیات از طریق منو در دسترس هستند.

● رمزگذاری

```
python main.py hill-encrypt --text "HELP" --key-matrix "3,3;2,5" --padding none
```

نتیجه:

```
HIAT
```

برای خواندن متن از فایل می‌توان به جای `--text` از `--input` استفاده کرد:

```
python main.py hill-encrypt --input data/sample_plaintext.txt --key-matrix "3,3;2,5" --output
output/ciphertext.txt
```

● رمزگشایی

```
python main.py hill-decrypt --text "HIAT" --key-matrix "3,3;2,5" --padding none
```

خروجی:

```
HELP
```

● پیدا کردن کلید از متن معلوم



```
python main.py hill-attack --plaintext "HELP" --ciphertext "HIAT" --block-size 2
```

خروجی:



```
Recovered key matrix: 3,3;2,5
```

فرمول رمز هیل

حروف انگلیسی به اعداد پیمانه ۲۶ تبدیل می‌شوند:

$A = 0, B = 1, \dots, Z = 25$

اگر اندازه بلوک n باشد، هر n حرف یک بردار ستونی P می‌سازد. کلید هم یک ماتریس مربعی K با اندازه $n \times n$ است. هر بلوک با فرمول زیر رمز می‌شود:

$C = K \times P \pmod{26}$

● مثال کامل همراه با پدینگ: رمزگذاری و رمزگشایی HELP

از نگاشت $A = 0, \dots, Z = 25$ و کلید زیر استفاده می‌کنیم:

$K = \begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$

چون اندازه کلید 2×2 است، هر بلوک دو حرف دارد. طول HELP برابر ۴ و از قبل مضرب ۲ است؛ باین‌حال در حالت `padding=length` یک بلوک کامل پدینگ اضافه می‌شود تا هنگام رمزگشایی حذف پدینگ بدون ابهام باشد. برای بلوک ۲تایی، B نشان می‌دهد که دو حرف پدینگ افزوده شده است:

`HELP → HELPB → HE | LP | BB`

بردار عددی بلوک اول، HE ، برابر $(7, 4)$ است:

$\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix} \times \begin{bmatrix} 7 \\ 4 \end{bmatrix} = \begin{bmatrix} 33 \\ 34 \end{bmatrix} = \begin{bmatrix} 7 \\ 8 \end{bmatrix} \pmod{26}$
 $(7, 8) \rightarrow HI$

بردار عددی بلوک دوم، LP ، برابر $(11, 15)$ است:



```
| 3 3 | × | 11 | = | 78 | = | 0 | (mod 26)
| 2 5 |   | 15 |   | 97 |   | 19 |

(0, 19) → AT
```

بردار عددی بلوک پدینگ، BB، برابر (1, 1) است:



```
| 3 3 | × | 1 | = | 6 | = | 6 | (mod 26)
| 2 5 |   | 1 |   | 7 |   | 7 |

(6, 7) → GH
```

پس متن رمز نهایی چنین است:



```
HELP → HELPPB → HI | AT | GH → HIATGH
```

برای رمزگشایی به وارون ماتریس کلید در پیمانه ۲۶ نیاز داریم. چون $\det(K)=9$ و $9^{-1} \equiv 3 \pmod{26}$ ، داریم:



```
K-1 = 3 × | 5 -3 | = | 15 17 | (mod 26)
          | -2 3 |   | 20 9 |
```

اکنون دو بلوک متن رمز را در K^{-1} ضرب می‌کنیم:



```
| 15 17 | × | 7 | = | 241 | = | 7 | (mod 26)
| 20 9 |   | 8 |   | 212 |   | 4 |

(7, 4) → HE

| 15 17 | × | 0 | = | 323 | = | 11 | (mod 26)
| 20 9 |   | 19 |   | 171 |   | 15 |

(11, 15) → LP

| 15 17 | × | 6 | = | 209 | = | 1 | (mod 26)
| 20 9 |   | 7 |   | 183 |   | 1 |

(1, 1) → BB
```

در نتیجه ابتدا متن پدشده `HELPBB` به دست می‌آید. حرف آخر `B` اعلام می‌کند که دو حرف پدینگ وجود دارد؛ برنامه هر دو `B` را حذف می‌کند و متن اصلی را برمی‌گرداند:



HIATGH → HE | LP | BB → HELPBB → HELP

● مثال 3×3 برای ترانزاده ماتریس هم‌عامل‌ها

در فرمول معکوس، خود ماتریس کلید ترانزاده نمی‌شود؛ ابتدا ماتریس هم‌عامل‌ها ساخته و سپس ترانزاده می‌شود:



$K^{-1} = \det(K)^{-1} \times \text{adj}(K) \pmod{26}$
 $\text{adj}(K) = \text{Cofactor}(K)^T$

برای نمونه، ماتریس کلید زیر را در نظر بگیرید:



$K = \begin{vmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{vmatrix}$

ماتریس هم‌عامل‌های آن برابر است با:



$\text{Cofactor}(K) = \begin{vmatrix} 70 & 5 & -99 \\ -343 & 70 & 378 \\ 224 & -47 & -216 \end{vmatrix}$

با ترانزاده کردن این ماتریس، ماتریس الحاقی به دست می‌آید:



$\text{adj}(K) = \text{Cofactor}(K)^T$
 $\text{adj}(K) = \begin{vmatrix} 70 & -343 & 224 \\ 5 & 70 & -47 \\ -99 & 378 & -216 \end{vmatrix}$

پیاده‌سازی به‌جای ساختن ماتریس هم‌عامل‌ها و ترانزاده کردن جداگانه آن، مستقیماً هم‌عامل خانه `[column][row]` را در خانه `[row][column]` خروجی قرار می‌دهد:

```
minor = matrix_minor(matrix, column, row)
```

در نتیجه، جای‌گذاری خانه‌ها چنین است:

خانه خروجی هم‌عامل استفاده‌شده مقدار پیش از ضرب در معکوس دترمینان

70	Cofactor[0][0]	[0][0]
343-	Cofactor[1][0]	[0][1]
224	Cofactor[2][0]	[0][2]
5	Cofactor[0][1]	[1][0]
70	Cofactor[1][1]	[1][1]
47-	Cofactor[2][1]	[1][2]
99-	Cofactor[0][2]	[2][0]
378	Cofactor[1][2]	[2][1]
216-	Cofactor[2][2]	[2][2]

برای مثال، هنگام محاسبه خانه $[0][1]$ خروجی، مقادیر حلقه $row=0$ و $column=1$ هستند. پس کد `matrix_minor(matrix, 1, 0)` را فراخوانی می‌کند؛ یعنی سطر ۱ و ستون ۰ ماتریس اصلی حذف می‌شوند:

```
minor = |24 1|
        |17 15|

det(minor) = 24×15 - 1×17 = 343
cofactor = (-1)^(0+1) × 343 = -343
```

بنابراین مقدار $Cofactor[1][0] = -343$ در خانه $[0][1]$ قرار می‌گیرد. این همان جابه‌جایی سطر و ستون در عمل ترانزاده است. چون $row + column$ با $column + row$ برابر است، علامت هم‌عامل نیز پس از جابه‌جایی تغییر نمی‌کند.

در این مثال:

```
det(K) = 441 ≡ 25 (mod 26)
det(K)^(-1) ≡ 25 (mod 26)
```

پس نتیجه نهایی برابر است با:



$$K^{-1} = 25 \times \text{adj}(K) \pmod{26}$$

$$K^{-1} = \begin{vmatrix} 8 & 5 & 10 \\ 21 & 8 & 21 \\ 21 & 12 & 8 \end{vmatrix}$$

شرط معتبر بودن کلید

ماتریس کلید باید در پیمانه ۲۶ وارون‌پذیر باشد. این شرط زمانی برقرار است که دترمینان ماتریس با ۲۶ نسبت به هم اول باشد:

```
gcd(det(K), 26) = 1
```

در ماتریس نمونه داریم:

```
det(K) = 3×5 - 3×2 = 9  
gcd(9, 26) = 1
```

پس معکوس پیمانه‌ای دترمینان وجود دارد و ماتریس برای رمز هیل قابل استفاده است. اگر این ب.م.م برابر ۱ نباشد، چند بردار مختلف ممکن است به یک بردار رمز یکسان برسند و رمزگشایی یکتا نخواهد بود.

الگوریتم اقلیدس تعمیم‌یافته و معکوس پیمانه‌ای

تابع `extended_gcd(first, second)` هم‌زمان ب.م.م دو عدد و ضریب بزو مربوط به عدد `first` را محاسبه می‌کند. این تابع در پروژه برای یافتن معکوس پیمانه‌ای دترمینان ماتریس کلید استفاده می‌شود.

فرض کنیم دو عدد ورودی الگوریتم `a` و `b` باشند. برای پیدا کردن معکوس پیمانه‌ای `a` نسبت به `b`، دنبال عددی مانند `x` هستیم که حاصل‌ضرب `a*x` پس از تقسیم بر `b` باقی‌مانده ۱ داشته باشد. الگوریتم اقلیدس تعمیم‌یافته ضرایبی مانند `x` و `y` پیدا می‌کند که:

$$ax + by = 1$$

عبارت `bxy` مضربی از `b` است؛ بنابراین باقی‌مانده تقسیم آن بر `b` همیشه صفر است. در نتیجه، برای اینکه مجموع بالا برابر ۱ شود، حاصل‌ضرب `ax` باید در پیمانه `b` باقی‌مانده ۱ داشته باشد:

$$ax \equiv 1 \pmod{b}$$

پس `x` معکوس پیمانه‌ای `a` نسبت به `b` است. برای مثال:

$$9 \times 3 + 26 \times (-1) = 1$$

جمله `26*(-1)` مضربی از ۲۶ است و در پیمانه ۲۶ برابر صفر می‌شود. بنابراین:

$$\begin{aligned} 9 \times 3 &= 27 \\ 27 \bmod 26 &= 1 \end{aligned}$$

پس معکوس پیمانه‌ای ۹ نسبت به ۲۶ برابر ۳ است. در رمز هیل، `a` همان `det(K)` و `b` برابر ۲۶ است.

کد تابع:

```
def extended_gcd(first, second):
    old_remainder = first
    remainder = second
    old_coefficient = 1
    coefficient = 0

    while remainder != 0:
        quotient = old_remainder // remainder
        old_remainder, remainder = (
            remainder,
            old_remainder - quotient * remainder
        )
        old_coefficient, coefficient = (
            coefficient,
            old_coefficient - quotient * coefficient
        )

    return old_remainder, old_coefficient
```

معنای متغیرهای تابع:

- **old_remainder**: عدد اول جفت فعلی الگوریتم اقلیدس؛ پس از پایان، آخرین باقی‌ماندهٔ غیرصفر و همان ب.م.م. است.
- **remainder**: عدد دوم جفت فعلی؛ در هر دور با باقی‌ماندهٔ تقسیم جایگزین می‌شود و با صفر شدن آن حلقه پایان می‌یابد.
- **quotient**: خارج‌قسمت `old_remainder // remainder` در دور جاری.
- **old_coefficient**: ضریب `first` در نمایش خطی `old_remainder`؛ پس از پایان، همان ضریب بزو موردنیاز است.
- **coefficient**: ضریب `first` در نمایش خطی `remainder`.

عبارت‌های زیر نمی‌گویند که باقی‌مانده به‌طور تصادفی یا بدون دلیل با ترکیبی از ۹ و ۲۶ برابر است. مفهوم آن‌ها این است که **هر باقی‌مانده‌ای که الگوریتم می‌سازد، یک ترکیب خطی از دو عدد اولیه است**. ترکیب خطی یعنی دو عدد اولیه را در ضرایب صحیح ضرب کنیم و نتیجه‌ها را با هم جمع کنیم.

در ابتدای مثال `extended_gcd(9, 26)` داریم:

```
old_remainder = 9 = 1×9 + 0×26
remainder     = 26 = 0×9 + 1×26
```

پس از تقسیم ۲۶ بر ۹، باقی‌ماندهٔ جدید ۸ است:

```

8 = 26 - 2×9
8 = (-2)×9 + 1×26

```

در مرحله بعد نیز باقی‌مانده جدید ۱ است:

```

1 = 9 - 8
1 = 9 - [(-2)×9 + 1×26]
1 = 3×9 - 1×26

```

بنابراین بله، باقی‌مانده‌ها واقعاً می‌توانند با این عبارتها برابر باشند؛ چون خود الگوریتم آنها را با تفریق مضرب‌هایی از باقی‌مانده‌های قبلی می‌سازد. ضرایب فقط ثبت می‌کنند که هر باقی‌مانده با چه ترکیبی از دو عدد اولیه ساخته شده است.

فرم کلی همین رابطه‌ها چنین است:

```

old_remainder = x1×first + y1×second
remainder     = x2×first + y2×second

```

در مثال `first=9` و `second=26`، این روابط به شکل زیر نوشته می‌شوند:

```

old_remainder = x1×9 + y1×26
remainder     = x2×9 + y2×26

```

در پیاده‌سازی فعلی فقط `x1` و `x2` ذخیره می‌شوند؛ یعنی `old_coefficient` و `coefficient`. ضرایب `y1` و `y2` ذخیره نمی‌شوند، چون برای یافتن معکوس پیمانه‌ای فقط ضریب عدد اول لازم است.

در هر دور، جفت باقی‌مانده‌ها از `(a, b)` به `(b, a - qb)` تبدیل می‌شود. این تبدیل ب.م.م را حفظ می‌کند و مقدار باقی‌مانده جدید از `remainder` کوچک‌تر است؛ بنابراین سرانجام `remainder` صفر و حلقه متوقف می‌شود.

● چرا ضرایب با همین فرمول تغییر می‌کنند؟

در هر دور، باقی‌مانده جدید چنین ساخته می‌شود:

```

new_remainder = old_remainder - quotient × remainder

```

برای نمونه `first=9` و `second=26` فرض می‌کنیم:



```
oldRemainder = x1×9 + y1×26
```

و:



```
remainder = x2×9 + y2×26
```

اگر `q` همان `quotient` باشد، باقی‌ماندهٔ جدید برابر است با:



```
newRemainder = oldRemainder - q×remainder
```

نمایش‌های قبلی را جای‌گذاری می‌کنیم:



```
newRemainder = [x1×9 + y1×26] - q[x2×9 + y2×26]
```

پرانتز را باز می‌کنیم:



```
newRemainder = x1×9 + y1×26 - q×x2×9 - q×y2×26
```

جمله‌های دارای ۹ و ۲۶ را کنار هم قرار می‌دهیم:



```
newRemainder = (x1 - q×x2)×9 + (y1 - q×y2)×26
```

پس ضریب جدید ۹ برابر است با:



```
x1 - q×x2
```

به همین علت کد ضریب جدید را با عبارت زیر محاسبه می‌کند:



```
old_coefficient - quotient * coefficient
```

● مثال کامل برای ۹ و ۲۶

برای ماتریس نمونه $\det(K)=9$ است؛ پس برنامه `extended_gcd(9, 26)` را اجرا می‌کند. وضعیت متغیرها چنین تغییر می‌کند:

مرحله	quotient	old_remainder	remainder	old_coefficient	coefficient
شروع —	9	26	1	0	0
دور ۱ 0	26	9	0	1	1
دور ۲ 2	9	8	1	2-	2-
دور ۳ 1	8	1	2-	3	3
دور ۴ 8	1	0	3	26-	26-

در مرحله سوم داریم:



```
26 = 2×9 + 8      → 8 = 26 - 2×9
9  = 1×8 + 1      → 1 = 9 - 8
```

با جای‌گذاری رابطه اول در رابطه دوم:



```
1 = 9 - (26 - 2×9)
  = 3×9 - 26
```

پس خروجی تابع `(1, 3)` است؛ عدد `1` ب.م.م و عدد `3` ضریب بزو برای ۹ است. در نتیجه:



```
9×3 ≡ 1 (mod 26)
```

بنابراین معکوس پیمانه‌ای ۹ نسبت به ۲۶ برابر ۳ است و برنامه از آن در فرمول $K^{-1} = \det(K)^{-1} \times \text{adj}(K) \pmod{26}$ استفاده می‌کند.

رمزگشایی و اثبات درستی آن

رمزگشایی با ماتریس معکوس انجام می‌شود:


$$P = K^{-1} \times C \pmod{26}$$

چون $K^{-1} \times K = I$ است، اگر مقدار C را از فرمول رمزگذاری جای‌گذاری کنیم، داریم:


$$\begin{aligned} & K^{-1} \times C \\ &= K^{-1} \times (K \times P) \\ &= (K^{-1} \times K) \times P \\ &= I \times P \\ &= P \pmod{26} \end{aligned}$$

بنابراین هر بلوکی که با یک کلید معتبر رمز شده باشد، با معکوس همان کلید دقیقاً بازیابی می‌شود.

کامل کردن بلوک‌ها (padding)

طول متن باید مضرب اندازه بلوک باشد. در حالت پیش‌فرض `length` برنامه اطلاعات padding را در انتهای متن آماده‌شده نگه می‌دارد تا هنگام رمزگشایی تعداد حروف اضافه مشخص باشد. گزینه `--padding none` هیچ حرفی اضافه نمی‌کند و فقط زمانی قابل استفاده است که طول متن از ابتدا مضرب اندازه بلوک باشد.

پیدا کردن کلید با متن معلوم

اگر چند بلوک متن اصلی را به صورت ستون‌های ماتریس P و بلوک‌های متن رمز متناظر را به صورت ستون‌های ماتریس C قرار دهیم، رابطه رمزگذاری چنین است:

$$C = K \times P \pmod{26}$$

اگر P در پیمانه ۲۶ وارون‌پذیر باشد، دو طرف را از راست در P^{-1} ضرب می‌کنیم:

$$C \times P^{-1} = K \times P \times P^{-1}$$

$$C \times P^{-1} = K \times I$$

$$K = C \times P^{-1} \pmod{26}$$

برنامه چند بلوک مناسب را انتخاب می‌کند، ماتریس کلید را می‌سازد و نتیجه را روی همه بلوک‌های داده‌شده بررسی می‌کند. اگر نتوان از بلوک‌های متن اصلی یک ماتریس وارون‌پذیر ساخت، کلید با این ورودی‌ها پیدا نمی‌شود.

نتیجه benchmark

پیاده‌سازی رمزگذاری، رمزگشایی و تولید گزارش به اندازه ثابت 2×2 یا 3×3 محدود نیست. برنامه اندازه کلید را از تعداد سطرهای ماتریس می‌گیرد و برای هر ماتریس مربعی معتبر کار می‌کند؛ برای نمونه، کلید 4×4 نیز در تست‌ها بررسی شده است. گزارش عملکردی که از منوی برنامه تولید می‌شود نیز همان کلیدی را آزمایش می‌کند که کاربر وارد کرده است و اندازه آن می‌تواند 2×2 ، 3×3 ، 4×4 یا اندازه معتبر دیگری باشد.

فایل‌های آماده موجود در پوشه `benchmarks/` با قابلیت عمومی برنامه تفاوت دارند: این فایل‌ها فقط خروجی یک اجرای ازپیش‌انجام‌شده برای کلیدهای 2×2 و 3×3 هستند. در آن اجرای ثابت، طول‌های ۱۰۰، ۵۰۰، ۱۰۰۰، ۳۰۰۰، ۵۰۰۰ و ۸۰۰۰ حرف هرکدام ۳۰ بار آزمایش شده‌اند. بنابراین ذکر 2×2 و 3×3 فقط اندازه‌های استفاده‌شده در آن benchmark ثبت‌شده را توصیف می‌کند، نه محدودیت کد را. اجرای معمولی برنامه این فایل‌های ثابت را تغییر نمی‌دهد؛ اما کاربر می‌تواند برای کلید دلخواه خود گزارش CSV و نمودار جدیدی در پوشه `output/` تولید کند.

اجرای تست‌ها



```
python -m unittest discover -s tests -v
```

تست‌ها مثال استاندارد [HELP → HIAT](#)، مثال ماتریس 3×3 ، معتبر بودن padding، رد ماتریس غیرقابل‌وارون و بازیابی کلید با متن معلوم را بررسی می‌کنند.

پایان گزارش بخش رمز هیل